Coordenação resiliente do despacho de ocorrências e da reserva de leitos de urgência

Arquitetura de Software: Teoria e Prática 2026 · Proposta do Grupo 3 · material para o grupo de desenvolvimento

Grupo 3: Felipe Melchior de Britto; Igor Benedito Costa Gonçalves; Sérgio Leandro Alves da Silva

Estudo de caso: Plataforma de Despacho de Ocorrências e Reserva de Leitos (SAMU / pronto-socorro)

Contextualização do problema

A regulação de urgência combina dois recursos escassos e disputados ao mesmo tempo: a ambulância que sai ao local e o leito que precisa existir no destino. O fluxo atravessa contextos distintos — ocorrência, frota, leito hospitalar e notificação — que não podem compartilhar a mesma transação nem o mesmo ciclo de implantação. Uma central monolítica acopla despacho, ocupação de leito e aviso ao hospital; uma divisão em serviços sem coordenação gera estados inválidos: ambulância em deslocamento sem leito; leito reservado em duplicidade para duas ocorrências; chamado aberto após a equipe já ter sido liberada; aviso que não chega à unidade de destino.

O problema de arquitetura é a coordenação transacional distribuída em um fluxo de baixa latência. Sem orquestração e compensação, falhas parciais (hospital indisponível, recusa de leito, timeout da equipe) deixam frota e leito inconsistentes. Consultas contínuas de ambulâncias livres e de leitos por especialidade, se forem sempre ao banco, degradam o tempo de despacho; cache sem invalidação faz a regulação escolher um recurso que já não existe. Sistemas externos (prontuário da unidade, canal de alerta) falham; sem resiliência no roteamento, a falha da unidade se propaga para a central e o chamado fica sem resposta.

O estudo de caso é uma plataforma própria de despacho e reserva de leito, no recorte de um consórcio municipal de urgência. Não se pretende um SAMU completo (geolocalização em tempo real, protocolo clínico, faturamento). O objetivo é tornar observável e tratável a inconsistência entre ocorrência, frota e leito em uma arquitetura de microsserviços.

Descrição da solução

A solução é uma arquitetura de microsserviços com orquestrador de Saga no despacho, mensageria para efeitos colaterais, cache da disponibilidade de frota e leitos, e API Gateway com políticas de resiliência.

O sistema de interesse é a Plataforma de Despacho de Ocorrências e Reserva de Leitos. O sistema superior é a rede municipal de urgência (central de regulação, bases de ambulância, unidades hospitalares e canais de alerta). Os sistemas subordinados e os atores externos estão no mapeamento de apoio.

Fluxo da Saga de despacho: (1) registrar ocorrência ABERTA e classificar a prioridade; (2) alocar ambulância DISPONIVEL; (3) reservar leito compatível na unidade de destino; (4) em sucesso, marcar ocorrência DESPACHADA, leito RESERVADO e publicar notificação à equipe e ao hospital; (5) em falha ou timeout, compensar (liberar ambulância, cancelar reserva de leito, reenfileirar o chamado ou cancelá-lo). Reservas de leito não confirmadas pela unidade após X minutos expiram por job agendado, reutilizando as mesmas compensações. A entrega das mensagens será at-least-once, com operações idempotentes (chave da ocorrência) para evitar dupla alocação de ambulância ou de leito.

Estudo de caso a ser desenvolvido: abertura de um chamado de urgência autenticado, duas ambulâncias e um conjunto limitado de leitos (com e sem vaga), hospital simulando aceite e recusa, e inspeção do estado final em ocorrência, frota, leito e notificação. Relatórios gerenciais, GIS e envio real de SMS ficam mockados. A interface será de menor esforço (OpenAPI/Swagger).

Atribuição por membro (aspecto técnico + funcionalidade na íntegra):

- **Membro 1 —** API Gateway, roteamento e resiliência (Resilience4j: timeout, retry, circuit breaker). Funcionalidade: autenticar regulador e abrir ocorrência.
- **Membro 2 —** Mensageria e padrão Saga (orquestração ocorrência–ambulância–leito). Funcionalidade: despachar ambulância e reservar leito, com compensação.
- **Membro 3 —** Cache de disponibilidade e agendamento. Funcionalidade: consultar/atualizar ocupação e expirar reservas não confirmadas.

Descrição das tecnologias

- **Java 21 e Spring Boot (Maven):** ecossistema obrigatório; base dos tutoriais hands-on (JPA, MongoDB, REST, OpenAPI, JUnit 5, Docker).
- **Spring Web + springdoc-openapi/Swagger:** APIs REST e demonstração das funcionalidades (menor esforço de UI).
- **Spring Data JPA + MySQL:** persistência de Paciente, Ocorrencia, Ambulancia, UnidadeSaude, Leito e Despacho. Resolve o armazenamento transacional local de cada serviço.
- **Spring Data MongoDB:** histórico de Notificacao e trilha de Auditoria do despacho. Resolve a separação do contexto de eventos em relação ao modelo relacional.
- **Spring AMQP (RabbitMQ):** filas de despacho, recusa de leito e alerta. Resolve o desacoplamento temporal dos passos da Saga.
- **Spring Data Redis:** cache de ambulâncias livres e de leitos por especialidade. Resolve a latência das consultas da regulação, com invalidação após alocar, confirmar ou compensar.
- **Spring Cloud Gateway:** ponto único de entrada da central. Resolve a composição da plataforma sem expor endereços internos à regulação e ao hospital.
- **Resilience4j:** tolerância a falhas na chamada ao MS-Leitos (unidade simulada indisponível). Resolve a propagação da instabilidade hospitalar para a central.
- **Spring Scheduler:** expiração de reservas não confirmadas. Resolve a devolução automática de leito e ambulância sem orquestrador externo.
- **JUnit 5:** testes de alocação exclusiva, idempotência e compensação.

Stack complementar: Docker Compose (MySQL, MongoDB, Redis, RabbitMQ). Ferramentas de IA generativa, se usadas, serão citadas na apresentação com demonstração breve de uso.

Material de apoio do grupo
Conteúdo de trabalho do grupo. Não faz parte do formulário de duas páginas entregue ao professor.

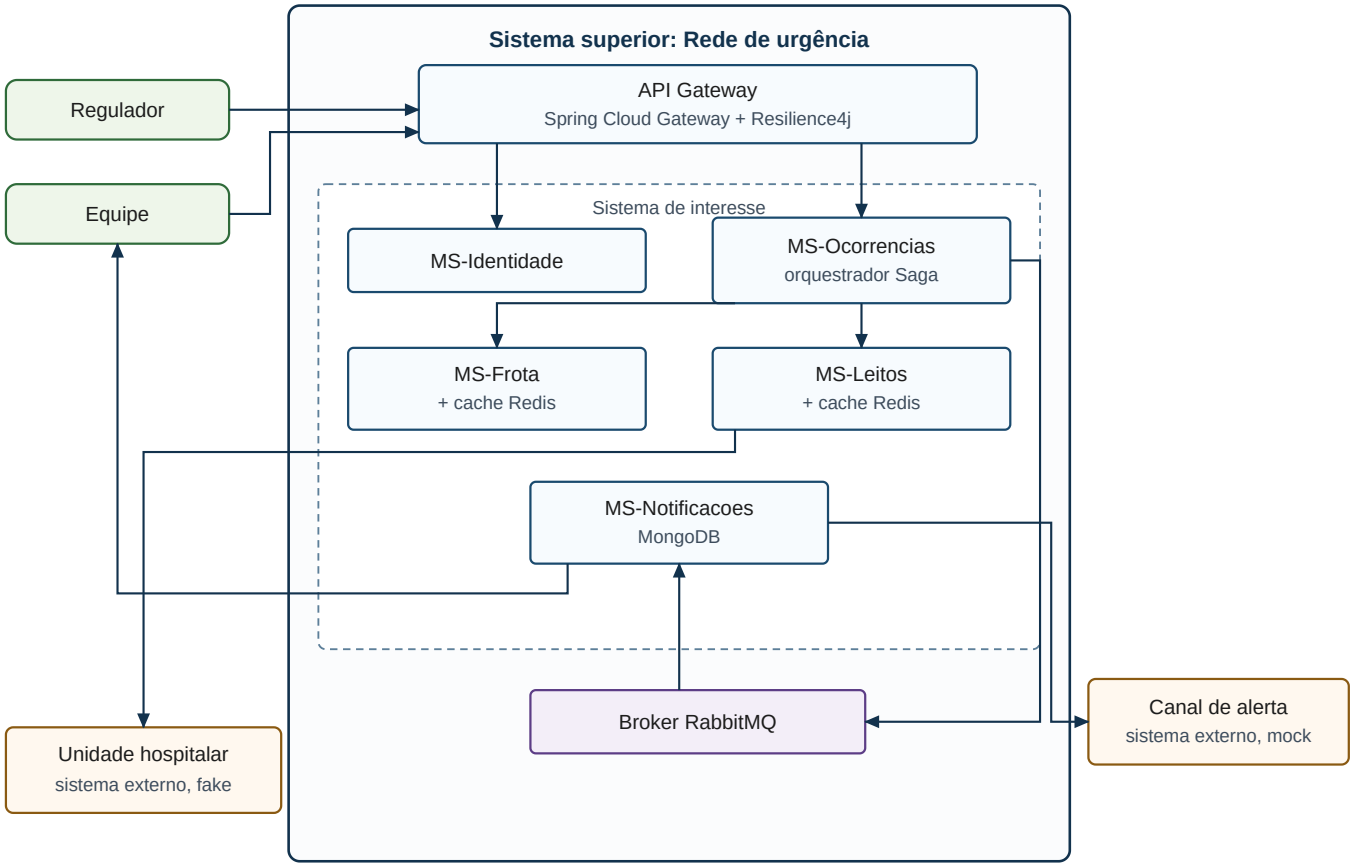
Por que esta proposta é nova

As especificações 01 (marketplace) e 02 (banco digital) foram usadas só como referência de forma: recursos disputados, baixa latência, filas, cache, gateway, agendamento e resiliência. O domínio, o fluxo de negócio e as entidades são outros: ocorrência de urgência, ambulância e leito hospitalar.

1. Mapeamento do modelo arquitetural

Sistema superior	Rede municipal de urgência e emergência (regulação, bases, hospitais, alertas).
Sistema de interesse	Plataforma de Despacho de Ocorrências e Reserva de Leitos.
Sistemas subordinados	MS-Identidade, MS-Ocorrencias, MS-Frota, MS-Leitos, MS-Notificacoes. MS-Auditoria/Relatorios permanece mockado.
Atores que interagem	Regulador (central), Equipe da ambulância, Unidade hospitalar (API fake de aceite/recusa de leito), Canal de alerta (SMS/push mockados), API Gateway.

2. Diagrama de contexto



Regulador e equipe entram pelo gateway. O MS-Ocorrencias orquestra frota e leitos. O MS-Leitos fala com a unidade hospitalar. O sucesso e os alertas passam pelo RabbitMQ até o MS-Notificacoes, que avisa a equipe e o canal de alerta.

3. Modelo de domínio (recorte)

```
enum Prioridade { VERMELHA, AMARELA, VERDE }
enum StatusOcorrencia { ABERTA, DESPACHADA, EM_TRANSPORTE, CONCLUIDA, CANCELADA }
enum StatusAmbulancia { DISPONIVEL, DESPACHADA, EM_ATENDIMENTO, INDISPONIVEL }
enum StatusLeito { LIVRE, RESERVADO, OCUPADO }
enum TipoLeito { UTI, EMERGENCIA, ENFERMARIA }

Paciente, Ocorrencia, Ambulancia, Equipe, UnidadeSaude, Leito, Despacho, Notificacao

Ocorrencia 1 -- 1 Paciente
```

```
Ocorrencia 1 -- 0..1 Despacho
Despacho 1 -- 1 Ambulancia
Despacho 1 -- 0..1 Leito
Ocorrencia 1 -- 0..* Notificacao
```

4. Funcionalidades na íntegra

Funcionalidade 1 — Abrir ocorrência de urgência

> Membro 1: Gateway e resiliência

1	O regulador acessa a plataforma	
2	O sistema solicita as informações de login	
3	O regulador informa usuário e senha	Entidade: Regulador
4	O sistema autentica e devolve o token	MS-Identidade.autenticar
5	O regulador seleciona “abrir ocorrência”	
6	O sistema exibe o formulário da ocorrência	
7	O regulador informa paciente, local e prioridade	Entidade: Paciente, Ocorrencia
8	O regulador confirma a abertura	
9	O Gateway encaminha a requisição ao MS-Ocorrencias (timeout / retry / circuit breaker)	
10	O sistema registra a ocorrência ABERTA	MS-Ocorrencias.abrir
11	O sistema exibe o identificador e o status	

Operações

- MS-Identidade: Token autenticar(String usuario, String senha)
- MS-Ocorrencias: Ocorrencia abrir(OcorrenciaRequest request)

Funcionalidade 2 — Despachar ambulância e reservar leito

> Membro 2: Saga e mensageria

1	O regulador solicita o despacho da ocorrência ABERTA	
2	O orquestrador localiza uma ambulância DISPONIVEL	Entidade: Ambulancia
3	O sistema aloca a ambulância	MS-Frota.alocar
4	O orquestrador solicita leito compatível no destino	Entidade: Leito
5	Em aceite, o leito fica RESERVADO e a ocorrência DESPACHADA	MS-Leitos.reservar MS-Ocorrencias.atualizarStatus
6	O orquestrador publica o evento OcorrenciaDespachada	
7	Em recusa, timeout ou falha, dispara compensação (liberar ambulância, cancelar reserva, reabrir ou cancelar o chamado)	MS-Ocorrencias.compensar
8	O sistema devolve o despacho e o status final	Entidade: Despacho

Operações

- MS-Frota: Ambulancia alocar(Long idOcorrencia, Prioridade prioridade)
- MS-Leitos: Leito reservar(Long idOcorrencia, TipoLeito tipo)
- MS-Ocorrencias: Ocorrencia atualizarStatus(Long id, StatusOcorrencia status)
- MS-Ocorrencias: void compensar(Long idOcorrencia)
- Broker: void publicar(EventoOcorrencia evento)

Funcionalidade 3 — Atualizar ocupação e expirar reserva não confirmada

> Membro 3: cache e agendamento

1	A regulação consulta ambulâncias livres e leitos por tipo	Entidade: Ambulancia, Leito
2	O sistema lê o cache (fallback ao banco)	MS-Frota.listarDisponiveis MS-Leitos.listarDisponiveis
3	Após alocar ou reservar, o cache é invalidado	
4	O job identifica reservas RESERVADO há mais de X minutos sem confirmação da unidade	
5	O sistema libera o leito e a ambulância	MS-Leitos.expirarReservas MS-Frota.liberar
6	O sistema atualiza a ocorrência	MS-Ocorrencias.atualizarStatus
7	O sistema enfileira alerta de expiração	MS-Notificacoes.enviar
8	O sistema registra a notificação	Entidade: Notificacao

Operações

- MS-Frota: List<Ambulancia> listarDisponiveis()
- MS-Frota: void liberar(Long idAmbulancia)
- MS-Leitos: List<Leito> listarDisponiveis(TipoLeito tipo)
- MS-Leitos: List<Leito> expirarReservas()
- MS-Notificacoes: Notificacao enviar(Long idOcorrencia, String mensagem)

5. Recorte

- Entrega de 16/09: tema + formulário (máx. 2 páginas).
- Projeto (outubro): as três funcionalidades acima ponta a ponta.
- Mockado: GIS/mapa, protocolo clínico, relatórios, envio real de SMS/push.
- Interface: Swagger.